

DETAILED PROJECT REPORT

PrepPilot

An Offline-First, AI-Assisted Study and Examination Preparation Platform

Prepared by

Archit Jain

October 2026

Abstract

PrepPilot is a mobile application that helps a student organise an examination syllabus, record what has genuinely been completed, time their study and revise, with or without an internet connection. A syllabus is modelled as a hierarchy of Subject → Chapter → Topic → optional Sub-topic, and every other feature — study timers, flashcards, notes, quizzes, reminders and an AI assistant — is attached to that hierarchy rather than living in a separate tool.

The application is offline-first: the SQLite database on the device is the source of truth, every read and write completes locally, and a synchronisation engine later reconciles the device with a Supabase (PostgreSQL) backend using client-generated UUIDv7 identifiers, tombstones and deterministic conflict rules. Progress is computed topic-weighted — a subject with one finished chapter of one topic and an unfinished chapter of ninety-nine topics is reported as 1% complete, not 50%. AI assistance is provided by Google Gemini through a server-side Edge Function, so the API key never reaches a device, and any syllabus change the assistant proposes must be reviewed and confirmed by the student before it is applied.

The system is built with React Native and Expo in TypeScript, as a monorepo of about 36,000 lines of non-test source. It is verified by an automated suite of 2,283 tests (2,231 executed for this report, all passing), has been built as an Android APK, and is also published as a browser demonstration. This report describes the problem, the requirements, the design and implementation, the evidence of testing, and the known limitations of the delivered system.

Keywords: offline-first, local-first software, study tracking, React Native, SQLite, Supabase, Row Level Security, synchronisation, Google Gemini, spaced repetition.

Table of Contents

1. Introduction..... 7
1.1 Project overview..... 7
1.2 Problem statement..... 7
1.3 Objectives..... 7
1.4 Scope..... 7
1.5 Organisation of the report..... 8
2. Literature Review and Existing Systems..... 8
2.1 The spacing effect..... 8
2.2 The SM-2 scheduling algorithm..... 8
2.3 Local-first software..... 8
2.4 Conflict resolution between replicas..... 8
2.5 Review of existing applications..... 8
3. Feasibility Study..... 9
3.1 Technical feasibility..... 9
3.2 Economic feasibility..... 9
3.3 Operational feasibility..... 9
4. Requirements Specification..... 9
4.1 Functional requirements..... 9
4.2 Non-functional requirements..... 10
4.3 Hardware and software requirements..... 10
5. System Design..... 11
5.1 Architecture..... 11
5.2 Working flow..... 11
5.3 Data model..... 12
5.4 Synchronisation design..... 13
5.5 AI assistant design..... 14
5.6 Security design..... 15
6. Implementation..... 15
6.1 Technology stack..... 15
6.2 Repository structure..... 16
6.3 Progress engine..... 16
6.4 Feature modules..... 16
6.5 Development timeline..... 17
7. User Interface and Screenshots..... 17
7.1 Syllabus tracker..... 18
7.2 Templates and building a course..... 19
7.3 Timers..... 19
7.4 Focus Mode..... 20
7.5 Dashboard and sessions..... 21
7.6 Revision, notes and reminders..... 21
7.7 AI assistant..... 22
7.8 Downloadable themes..... 23

8. Testing and Validation **24**
8.1 Strategy 24
8.2 Test results 24
8.3 What is tested against something real 24
8.4 Continuous integration 24
8.5 Device testing 24
9. Results and Discussion **24**
10. Limitations **25**
11. Future Scope **25**
12. Conclusion **25**
References **26**
Appendix A: Glossary **26**

List of Figures

Figure 5.1: System architecture....11
Figure 5.2: Every feature attaches to the syllabus hierarchy....11
Figure 5.3: Working flow of the system....12
Figure 5.4: Entity relationships of the core syllabus tables (notes, chats and quizzes, added later, are listed in Table 5.1)....13
Figure 5.5: Offline write and later synchronisation....14
Figure 5.6: Conflict resolution decision flow....14
Figure 5.7: AI request flow through the Supabase Edge Function....15
Figure 7.1: Tracker with the JEE Main 2026 template....
Figure 7.2: A subject expanded into chapters....
Figure 7.3: Chapter expanded into topics....
Figure 7.4: Topic options menu....
Figure 7.5: Quiz settings: count, difficulty, multi-answer....
Figure 7.6: Tracker in light mode....
Figure 7.7: Template library....19
Figure 7.8: Template preview before import....
Figure 7.9: Build a course from scratch....
Figure 7.10: Timer sheet....19
Figure 7.11: Pomodoro mode before starting....
Figure 7.12: Pomodoro interval running....
Figure 7.13: Full-screen timer, Flip cards face (paused)....
Figure 7.14: Minimal face....20
Figure 7.15: Ring face....20
Figure 7.16: Time wedge face....20
Figure 7.17: Hourglass face....20
Figure 7.18: Rising tide face....20
Figure 7.19: Focus Mode set-up....20
Figure 7.20: End alert and capability list....
Figure 7.21: Focus Mode....20
Figure 7.22: Dashboard....21
Figure 7.23: Sessions history....21
Figure 7.24: Sessions with Quiz deck, light mode....
Figure 7.25: Flashcard review with 1d / 2d / 4d....
Figure 7.26: Notes by topic....21
Figure 7.27: Reminders....21
Figure 7.28: Attach a photo or screenshot....
Figure 7.29: Picture attached to a question....
Figure 7.30: Answer about the picture....
Figure 7.31: Formatted reply....
Figure 7.32: Proposal review before adding....
Figure 7.33: Settings....22
Figure 7.34: Theme list in Settings....
Figure 7.35: Notebook theme, dark....
Figure 7.36: Notebook dashboard, dark....

Figure 7.37: Flight deck theme, dark.....
Figure 7.38: Bold bento theme, dark.....
Figure 7.39: Notebook theme, light.....
Figure 7.40: Flight deck theme, light.....
Figure 7.41: Bold bento theme, light.....

List of Tables

Table 2.1: Comparison with existing applications.....8
Table 3.1: Development and running cost.....9
Table 4.1: Functional requirements and delivery status.....9
Table 4.2: Non-functional requirements.....10
Table 4.3: Development and runtime environment.....10
Table 5.1: Database tables.....12
Table 5.2: Conflict resolution rules.....13
Table 6.1: Technology stack.....15
Table 6.2: Repository layout and size (non-test source lines, 6 Oct 2026).....16
Table 6.3: Feature modules.....16
Table 6.4: Milestones, from the repository history and decision log.....17
Table 8.1: Automated test results.....24
Table 9.1: Objectives against outcomes.....24

1. Introduction

1.1 Project overview

PrepPilot is an offline-first, AI-assisted study tracking application for students preparing for school board examinations, competitive entrance examinations and professional qualifications. Its organising idea is that a syllabus is a structure and progress is a measurement over that structure. The student loads a syllabus — from one of 22 bundled templates or built from scratch — ticks off topics as they are genuinely finished, times study sessions against a subject, chapter or topic, and sees honest progress figures on the tracker, the dashboard and the sessions history.

The guiding principle of the product is **Tracker first, AI second.** The tracker is complete and useful without any network connection; the Gemini-powered assistant supports the workflow — explaining, summarising, quizzing and suggesting additions — but never drives it and can never change the syllabus without the student's explicit confirmation. PrepPilot is a tool for individual self-improvement and has no social features.

Android is the primary platform. The same codebase is exported to the web with `react-native-web` as a demonstration build, hosted at `https://preppilottdemo.netlify.app`.

1.2 Problem statement

Students commonly use several unrelated applications for syllabus tracking, timing, reminders, revision and AI help, none of which holds the syllabus they should all refer to. A CBSE Class 12, JEE Main or UPSC Civil Services syllabus contains hundreds of individually learnable topics; paper cannot compute a percentage, spreadsheets are laborious on a phone, and memory is least reliable when it matters most. Study also frequently happens without connectivity — in libraries, hostels and on trains — so a tool that needs a network round-trip to tick a box fails exactly where it is needed.

A more serious problem is inaccurate measurement. Averaging chapter percentages — the obvious implementation — reports a subject with a 1-of-1 chapter and a 0-of-99 chapter as (100% + 0%) ÷ 2 = 50% complete, when the truthful figure is 1 ÷ 100 = 1%. A student who believes they are halfway through a subject they have barely begun will plan their remaining weeks wrongly. PrepPilot therefore treats correct measurement as its central obligation.

1.3 Objectives

1. **Accurate measurement** — model a syllabus as a hierarchy and compute progress topic-weighted; report an empty portion as unmeasured (—), never as 0%.
2. **Offline-first data** — every read and write completes against the device database; changes on several devices reconcile later by a deterministic rule.
3. **Safe AI** — keep the AI credential server-side and require human review and confirmation for any change the assistant proposes.
4. **One integrated workflow** — bind timers, flashcards, notes, quizzes and reminders to the same syllabus so effort, revision and progress stay connected.

1.4 Scope

In scope: an Android application and a browser demonstration build covering syllabus tracking, progress, timers and study sessions, dashboard and history, templates and a course builder, multiple courses, notes, flashcards, quizzes, reminders, Focus Mode, downloadable themes, an AI assistant, authentication, cloud synchronisation and account deletion.

Out of scope: social or collaborative features, teacher or institution dashboards, an iOS release, blocking other applications (not permitted on Android without restricted privileges) and payments.

1.5 Organisation of the report

Chapter 2 reviews related work and existing applications. Chapter 3 assesses feasibility. Chapter 4 specifies requirements. Chapter 5 presents the system design and Chapter 6 its implementation. Chapter 7 walks through the user interface with screenshots. Chapter 8 reports testing, Chapter 9 results, Chapter 10 limitations and Chapter 11 future scope; Chapter 12 concludes.

2. Literature Review and Existing Systems

2.1 The spacing effect

Ebbinghaus [1] showed that retention decays predictably over time and that review restores it. Cepeda et al. [2] synthesised distributed-practice studies and confirmed that material reviewed at spaced intervals is retained substantially better than material reviewed in a single massed session. This justifies a spaced review facility and informs PrepPilot's flashcard module.

2.2 The SM-2 scheduling algorithm

Wozniak and Gorzelanczyk [3] proposed SM-2, which keeps an ease factor and interval per item, lengthening the interval on successful recall and resetting it on failure; it underlies widely used software such as Anki. PrepPilot implements SM-2 in full and keeps its statistics up to date, but its review screen deliberately offers three fixed intervals (1, 2 and 4 days) so that each button says exactly what it will do. The scheduling PrepPilot performs today is therefore fixed-interval spaced repetition, not SM-2 scheduling.

2.3 Local-first software

Kleppmann et al. [4] argue that applications should keep primary data on the user's own device while still supporting synchronisation, and set out ideals including that the network should be optional. PrepPilot follows this directly: the device database is authoritative and the server is a copy.

2.4 Conflict resolution between replicas

Shapiro et al. [5] introduced Conflict-free Replicated Data Types (CRDTs), which converge without coordination but carry implementation and storage overhead. For PrepPilot's data — independently owned fields, infrequent edits — last-write-wins with tombstones, plus a separate timestamp for topic completion, converges deterministically at far lower cost. The review therefore informed a deliberate decision not to adopt CRDTs.

2.5 Review of existing applications

Table 2.1: Comparison with existing applications

Application type	Example	Syllabus structure	Weighted progress	Works offline	Study tooling
Flashcard apps	Anki, Quizlet	No	No	Yes (Anki)	Revision only
Task managers	Todoist	Flat lists	No	Partly	None
Focus timers	Forest (Pomodoro [6])	No	No	Yes	Time only
Coaching portals	Institute apps	Their own	Varies	No	Varies
PrepPilot	—	4 levels, any syllabus	Topic-weighted	Yes	Timers, cards, notes, quizzes, AI

No reviewed application combines a structured syllabus, an honest progress calculation, offline-first operation and integrated study tooling. That gap is what PrepPilot addresses.

3. Feasibility Study

3.1 Technical feasibility

Expo provides a mature cross-platform runtime with modules for local SQLite storage, secure credential storage, notifications, image picking and screen orientation, while Supabase supplies managed PostgreSQL, authentication, file storage and serverless Edge Functions, so no custom server has to be operated. The principal technical risks were addressed in the design: identifier collisions on offline creation are prevented by client-generated UUIDv7 keys, lost deletions by tombstones, and device clock skew by rewinding the synchronisation watermark. Two platform limits are genuine and are stated in the app: Android does not let one app block another without privileges restricted by Play Store policy, and no available module exposes the permission needed to toggle Do Not Disturb.

3.2 Economic feasibility

Every framework and library used is open source, and Supabase and Gemini both offer free tiers adequate for development and demonstration. The only cost that grows with use is the Gemini API. The figures below are the project's own estimates from what the app sends, using Google's published paid-tier prices as checked on 24 September 2026; they should be re-checked before being relied on.

Table 3.1: Development and running cost

Item	Cost	Notes
React Native, Expo, SQLite, Drizzle ORM, PostgreSQL	Free	Open-source licences
Supabase (database, auth, storage, functions)	Free tier	Adequate at project scale
Gemini API — free tier	Free	~20 requests/day on the main model, ~500 on Flash-Lite, shared by all users; content may be used by Google
Gemini API — paid tier, per busy student	≈ US$2.40 / month	20 questions + 2 quizzes a day on Gemini 3.6 Flash at 2026 prices; about half on Flash-Lite
Expo Application Services (APK builds)	Free tier	Used to build the Android package
Hosting of the web demo (Netlify)	Free tier	Static files only

Uncontrolled spending is bounded by a per-account limit of 20 AI requests an hour in the Edge Function. That limit is held in memory per function instance, so it bounds runaway use rather than acting as a durable quota.

3.3 Operational feasibility

The primary interaction — expand a subject, tick a topic — needs no instruction. Because the app works without connectivity it places no dependency on an institution's network, and bundled templates remove most of the initial setup, which is where such tools are usually abandoned. Synchronisation runs automatically, so the student never has to remember to back up.

4. Requirements Specification

4.1 Functional requirements

Table 4.1: Functional requirements and delivery status

ID	Requirement	Delivered as

FR-1	Authentication	Email and password with confirmation link, Forgot password reset link, native Google sign-in, guest use; phone OTP implemented behind a flag
FR-2	Syllabus management	Create, rename, reorder and delete subjects, chapters, topics and optional sub-topics; several courses
FR-3	Topic completion and progress	Checkboxes; topic-weighted progress for chapter, subject, course
FR-4	Timers and study sessions	Stopwatch, Pomodoro (25/5/15 min, long break every 4) and custom; full-screen landscape timer with six faces
FR-5	History and statistics	Dashboard: completion, today, total, streak, last 7 days, time by subject; Sessions history
FR-6	Templates and custom syllabus	22 bundled templates plus 2 examples; preview, pick subjects, import; Build from scratch
FR-7	Revision	Flashcards with 1d / 2d / 4d review; quizzes of 1–15 questions at four difficulties saved to a Quiz deck
FR-8	Notes	Any number of notes per topic or sub-topic, including saved AI summaries and replies
FR-9	Reminders	Student-created weekday and one-off reminders
FR-10	Focus Mode	1–240 min, up to six presets, end alert choice, keep-screen-on, pause
FR-11	AI assistance	Ask, summarise, quiz, photo questions, saved chats, follow-ups, propose-then-confirm additions
FR-12	Offline operation and sync	SQLite on device; Supabase sync on sign-in, every 5 min, on leaving the app, on reconnecting
FR-13	Themes	Classic built in; Notebook, Flight deck and Bold bento downloaded on demand
FR-14	Account deletion	Removes local data and, through an Edge Function, the server account

4.2 Non-functional requirements

Table 4.2: Non-functional requirements

Quality	How it is met
Offline capability	All reads and writes go to local SQLite; sync is a separate, later step
Correctness	Progress, timer, scheduling and merge logic are pure functions in a shared package, tested exhaustively
Security	Row Level Security on every table; server-only AI key; sessions in the device keystore; validated input
Performance	The tracker reads a course in four queries; sync pulls in pages of 1,000 rows
Accessibility	Tests query by accessibility role and name, so unreachable controls fail the suite; theme packs checked for WCAG contrast
Maintainability	TypeScript throughout; decision log (entries D1–D110); CI on every change; coverage thresholds
Portability	One codebase for Android and the browser

4.3 Hardware and software requirements

Table 4.3: Development and runtime environment

Category	Requirement
Development machine	At least 8 GB RAM; Windows, macOS or Linux
Test device	Physical Android phone, Android 8.0 or later (notifications, keep-awake, keystore and app-state changes cannot be verified reliably in an emulator or browser)
Runtime and tools	Node.js 22, pnpm 10+, Expo CLI and EAS, Supabase CLI, Deno (Edge Functions), Git and GitHub
Cloud accounts	Supabase project; Google AI Studio key for Gemini; Google Cloud OAuth clients for sign-in
End user	Android phone; internet needed only for first sign-in, sync, AI and theme downloads

5. System Design

5.1 Architecture

The client is layered. Expo Router screens and design-system components form the presentation layer; one Zustand store per feature holds state; repositories give typed access to SQLite through Drizzle ORM; and a framework-independent domain package (`packages/shared`) holds the progress engine, timer, scheduling, merge rules, AI parsing and design tokens. Because the Android app and the web build use the same domain package and tokens, they cannot disagree about a percentage or a colour. The backend is Supabase: PostgreSQL with Row Level Security, authentication, a public read-only Storage bucket for themes, and two Deno Edge Functions, `ask-gemini` and `delete-account`.

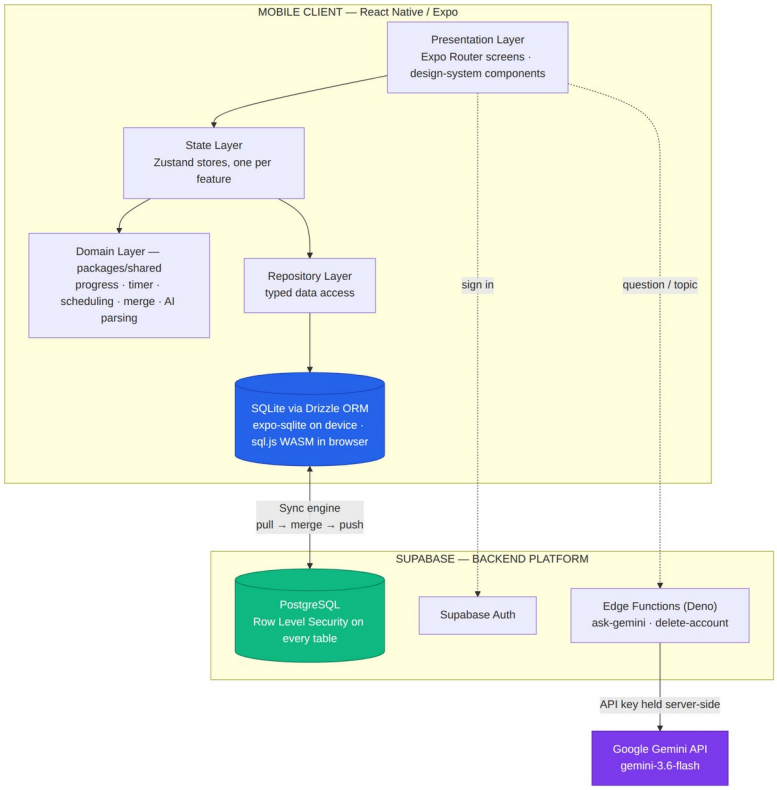


Figure 5.1: System architecture

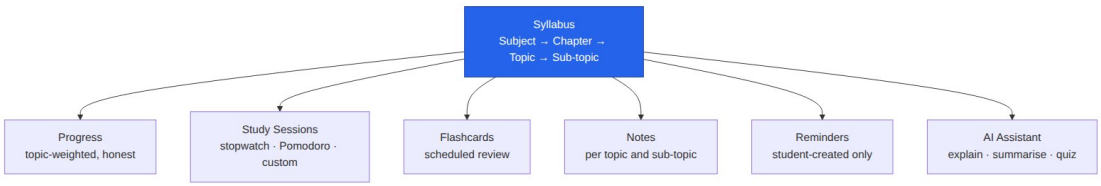


Figure 5.2: Every feature attaches to the syllabus hierarchy

5.2 Working flow

A student signs in, imports or builds a syllabus and works against it. Every action writes to the local database immediately and is marked pending; the synchronisation engine later pulls remote changes, merges them and pushes the agreed result. No student action waits on the network.

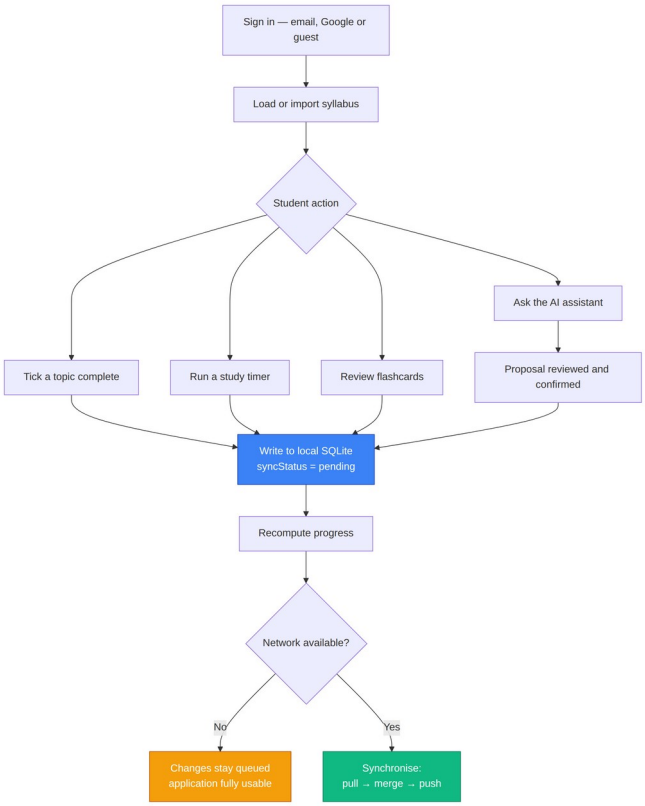


Figure 5.3: Working flow of the system

5.3 Data model

The device holds 13 SQLite tables; the server mirrors the 12 synchronised ones and adds `profiles`. Every synchronised row carries a client-generated UUIDv7 `id`, a denormalised `user_id`, `created_at`, `updated_at`, a `deleted_at` tombstone and, locally only, a `sync_status` of pending or synced.

Table 5.1: Database tables

Table	Purpose	Synced
trackers	Courses (a student may have several)	Yes
subjects, chapters, topics, subtopics	The syllabus hierarchy; topics carry a second timestamp for completion	Yes
study_sessions	Timed sessions; no foreign keys and name snapshots so history survives deletes and renames	Yes
flashcards	Cards with due date and SM-2 statistics	Yes
notes	Any number per topic; source manual, summary or assistant	Yes
reminders	Student reminders; the scheduled notification id stays local	Yes
chat_threads, chat_messages	Saved assistant conversations; pictures stay on the device	Yes
quiz_attempts	Finished quizzes with settings, answers and score	Yes
preferences	Device settings, including the sync watermark	No
profiles	Server only; created by a trigger on sign-up	Server

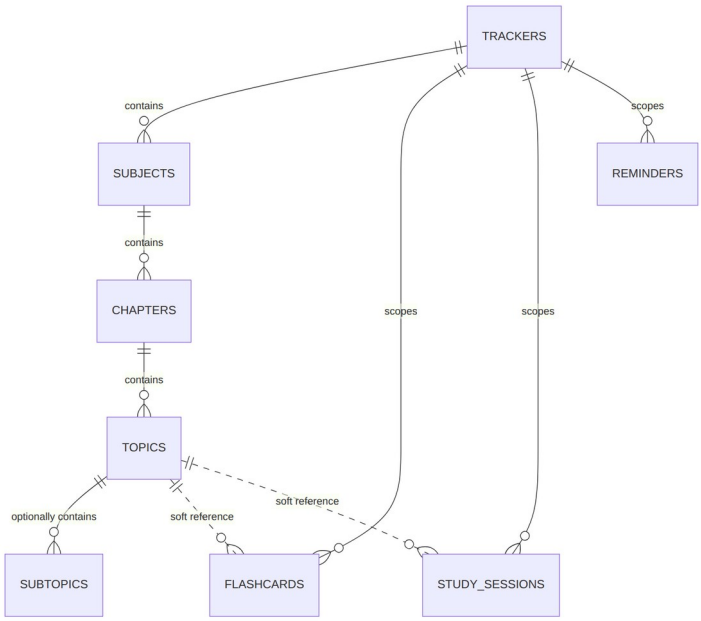


Figure 5.4: Entity relationships of the core syllabus tables (notes, chats and quizzes, added later, are listed in Table 5.1)

5.4 Synchronisation design

For each table, in dependency order so that a chapter never arrives before its subject, the engine pulls rows changed since a watermark, reconciles them with local rows by id, merges conflicts and writes the winner locally, then pushes everything still pending. Pulling before pushing is essential: pushing first would upload an unreconciled row and silently overwrite a change made elsewhere. The watermark is rewound 60 seconds on each pull to absorb clock differences and advanced only after a successful pass; every write is an idempotent upsert, so a failed pass can always be repeated.

Table 5.2: Conflict resolution rules

Situation	Winner	Reason
Either side tombstoned	The tombstone	Resurrecting a deleted row is worse than losing an edit
topics.completed differs	Later completed_changed_at	A rename on one device must not undo a tick on another
Completion timestamps equal	Completed	Un-ticking finished work is the least forgivable outcome
Anything else	Later updated_at	Last write wins
updated_at exactly equal	Local	Prevents devices rewriting rows they agree on

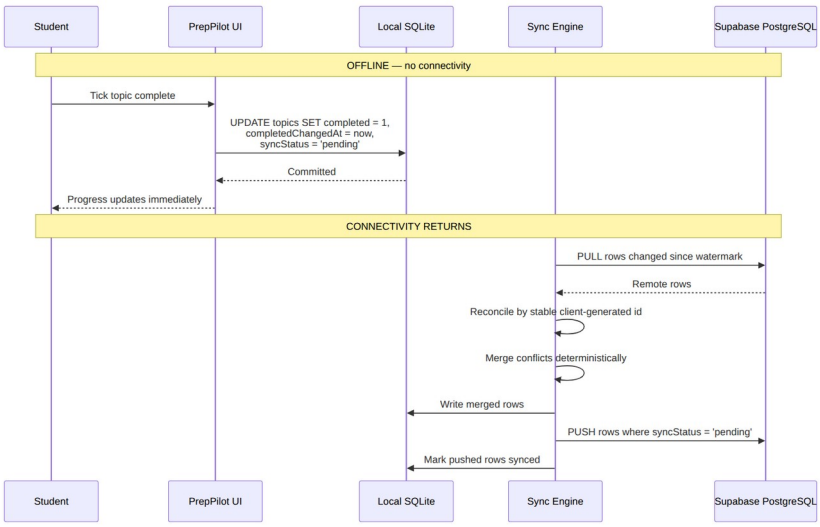


Figure 5.5: Offline write and later synchronisation

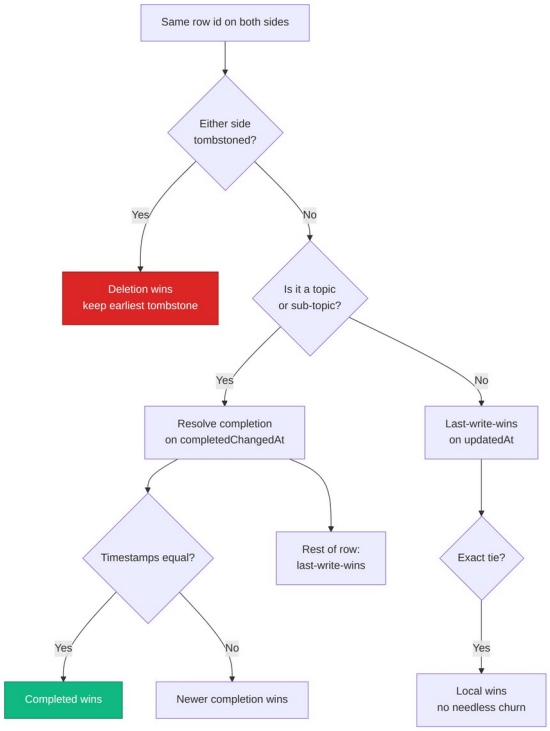


Figure 5.6: Conflict resolution decision flow

5.5 AI assistant design

The app calls the `ask-gemini` Edge Function, which holds the Gemini key as a Supabase secret, applies the per-user rate limit, and never forwards an upstream error body. It asks Gemini 3.6 Flash, retries once on 429 or 503, then falls back to Gemini 3.5 Flash-Lite and Gemini 3.1 Flash-Lite. A reply that suggests syllabus changes is parsed into an inert proposal whose grammar has only two verbs, `addChapters` and `addTopics`; there is no way to express deleting or completing anything. The student sees exactly what would be added and must confirm, and proposals over 200 topics are refused.

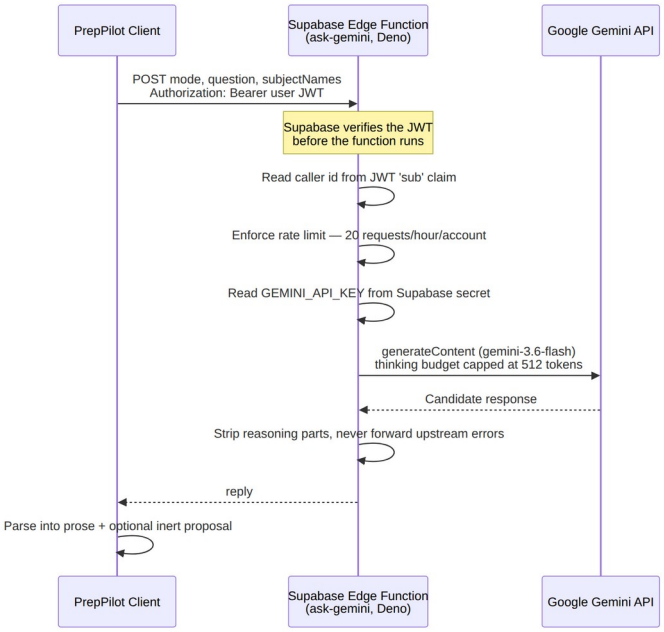


Figure 5.7: AI request flow through the Supabase Edge Function

5.6 Security design

- **Row Level Security** is enabled and forced on every table, every policy is scoped to `auth.uid()`, and no delete privilege is granted anywhere — rows are tombstoned.
- **Secrets:** only the publishable Supabase key is in the app; the service-role key is nowhere in the repository and the Gemini key exists only as a server secret.
- **Sessions** are stored in the device keystore through `expo-secure-store`, with an adapter that chunks values over its 2,048-byte limit.
- **Sign-in** uses PKCE for email links and password reset, and Google ID tokens from Android's account picker verified by Supabase.
- **Input validation** happens before any request; passwords are capped at 72 bytes because bcrypt ignores everything after that.
- **Errors never leak:** tests assert that keys, constraint names and driver errors never appear on screen.

6. Implementation

6.1 Technology stack

Table 6.1: Technology stack

Layer	Technology
Language	TypeScript
Mobile framework	React Native 0.86, Expo SDK 57, Expo Router
State	Zustand
Local database	SQLite via Drizzle ORM — expo-sqlite on Android, sql.js (WebAssembly) in the browser
Backend	Supabase: PostgreSQL, Auth, Storage, Edge Functions (Deno)
AI	Google Gemini 3.6 Flash, with 3.5 and 3.1 Flash-Lite fallbacks
Testing	Jest with React Native Testing Library (app), Vitest (shared), Deno test (functions)
Build and CI	pnpm workspaces, GitHub Actions, Expo Application Services, Netlify

6.2 Repository structure

Table 6.2: Repository layout and size (non-test source lines, 6 Oct 2026)

Path	Contents	Lines
apps/mobile	The Expo app: screens, features, stores, repositories, web build	25,138
packages/shared	Progress engine, timer, scheduling, sync merge, AI parsing, templates, theme validation, tokens	10,482
supabase/functions	Edge Functions ask-gemini and delete-account	773
supabase/migrations	PostgreSQL schema, policies and grants	832
docs	Product, technical and developer documentation; this report	—

6.3 Progress engine

Progress is the product's core promise, so it lives in one pure module and is tested exhaustively. Subject progress sums completed and total topic units across all of the subject's chapters, rather than averaging chapter percentages; a topic split into sub-topics contributes its sub-topics. An explicit regression test fails if averaging is ever introduced. The excerpt below is from `packages/shared/src/progress/progress.ts`.

```
export function makeProgress(completed: number, total: number): Progress {
  ...
  return {
    completed,
    total,
    percent: total === 0 ? null : (completed / total) * 100,
    isEmpty: total === 0,
  };
}

export function calculateSubjectProgress(subject: SubjectLike): Progress {
  let completed = 0;
  let total = 0;
  for (const chapter of subject.chapters) {
    for (const topic of chapter.topics) {
      const units = topicUnits(topic);
      completed += units.completed;
      total += units.total;
    }
  }
  return makeProgress(completed, total);
}
```

Rounding is also honest: 9,999 of 10,000 topics displays as 99%, never 100%, and a chapter with no topics shows — instead of 0%.

6.4 Feature modules

Table 6.3: Feature modules

Module	Implementation notes
Tracker	Expandable subjects and chapters, per-level progress rings and bars, play button per row, options menu (rename, summarise, test me, sub-topic, note, move, delete)
Timers	Stopwatch, Pomodoro and custom modes from a pure timer state machine; Pomodoro breaks are excluded from study time; full-screen landscape view with Flip cards, Minimal, Ring, Time wedge, Hourglass and Rising tide faces
Focus Mode	Configurable countdown; an in-app capability list states what the platform does and does not allow
Templates	22 syllabi: JEE Main, JEE Advanced, NEET UG; ten CBSE streams (classes 9–12); UPSC CSE, SSC CGL, GATE CS & IT, NEET PG, CA Foundation, Intermediate and Final, MBBS, DevOps

Flashcards	Review with 1d / 2d / 4d buttons; SM-2 statistics maintained underneath
Quizzes	1–15 multiple-choice questions, Easy / Medium / Hard / Competitive, optional multi-answer; each question validated separately; finished quizzes saved to the Quiz deck and can be redone offline
AI assistant	Photo questions resized on the device to 1,600 px JPEG; Markdown and LaTeX rendered as formatted text by an in-house parser; saved chats; follow-ups that send the summary and last six messages
Themes	JSON packs and fonts validated whole before use (colours, ranges, font paths, WCAG contrast), downloaded to the device, restored before the first frame
Sync	Runs on sign-in and launch, every 5 minutes in the foreground, on leaving the app, on reconnecting, and from Settings

6.5 Development timeline

Table 6.4: Milestones, from the repository history and decision log

Date (2026)	Milestone
30 Aug	Repository history begins
10 Sep	Repository audit, synopsis and first project presentation added
12 Sep	APK confirmed working on a physical Android phone against the live backend
21 Sep	All tables live on the hosted Supabase project; reinstall restores data
23 Sep	Native Google sign-in configured on the live project
24 Sep	Several notes per topic, saved chats, configurable quizzes and the Quiz deck
25 Sep	Follow-up questions on summaries; second Flash-Lite fallback
26 Sep	Forgot password and branded authentication emails
2 Oct	Downloadable themes: Notebook, Flight deck, Bold bento
3 Oct	Full-screen landscape timer and configurable Focus Mode
4 Oct	Continue-tracking card removed; web build deployable to Netlify

7. User Interface and Screenshots

This chapter walks through the application screen by screen. Screenshots come from two sources, named under each one: screens **captured 24 Sep 2026** are the set already kept in the repository (`docs/assets/screenshots/app`); screens marked **web demo, 6 Oct 2026** were captured for this report from the browser build of the same code at a phone-sized viewport, to cover features added since then. The browser build runs in demo mode with a small seeded syllabus, so its figures (such as 10.00%) are sample data. Sign-in, registration and password-reset screens are not shown because demo mode bypasses authentication.

7.1 Syllabus tracker

The tracker is the home screen: the course selector, overall completion ring, and one card per subject with its own progress ring and a play button that starts a timer. Expanding a subject shows chapters with progress bars, and expanding a chapter shows topics with checkboxes.

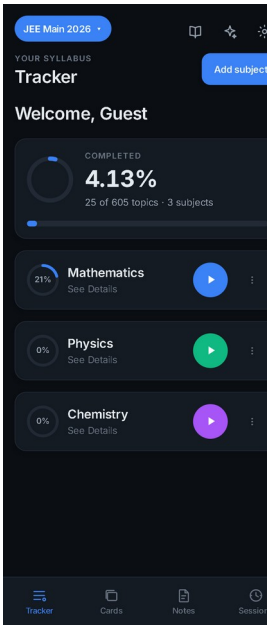


Figure 7.1: Tracker with the JEE Main 2026 template
captured 24 Sep 2026

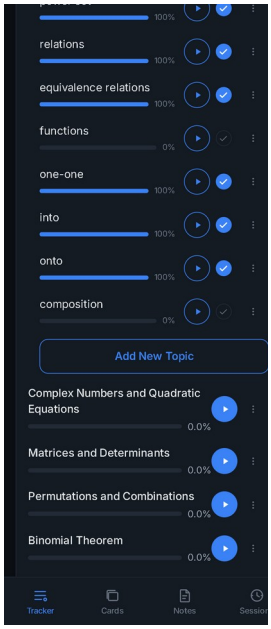


Figure 7.2: A subject expanded into chapters
captured 24 Sep 2026

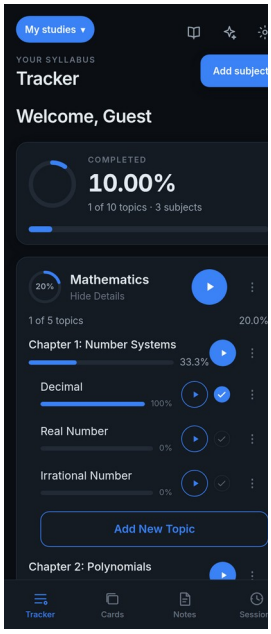


Figure 7.3: Chapter expanded into topics
web demo, 6 Oct 2026

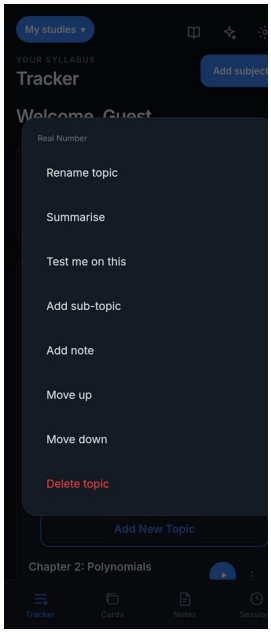


Figure 7.4: Topic options menu
web demo, 6 Oct 2026

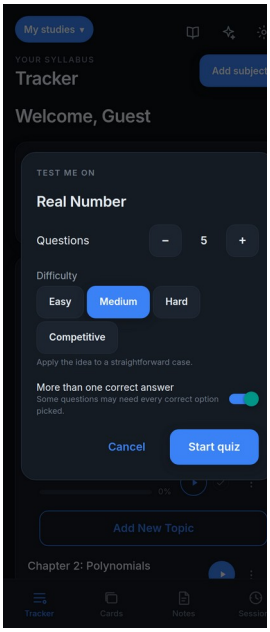


Figure 7.5: Quiz settings: count, difficulty, multi-answer
web demo, 6 Oct 2026

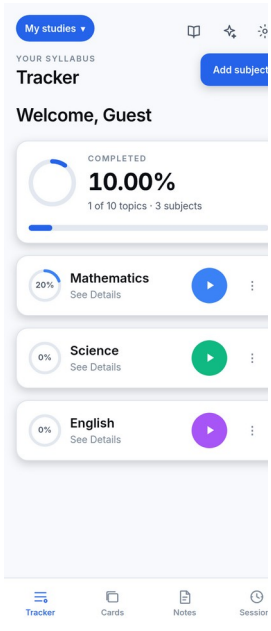


Figure 7.6: Tracker in light mode
web demo, 6 Oct 2026

7.2 Templates and building a course

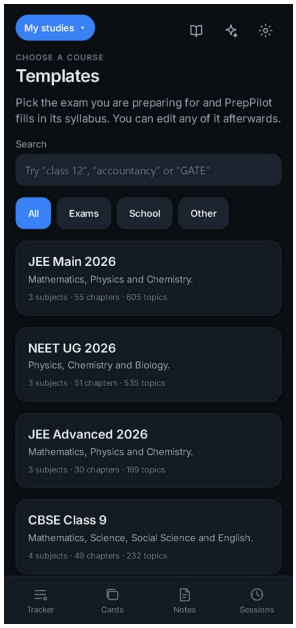


Figure 7.7: Template library
captured 24 Sep 2026

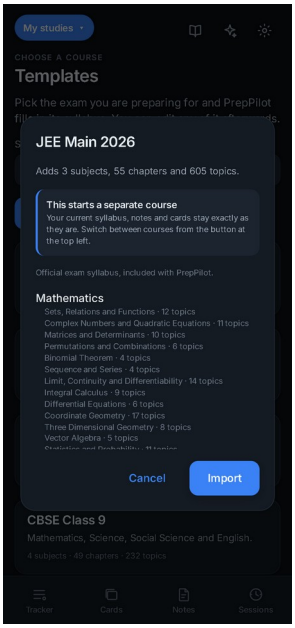


Figure 7.8: Template preview before import
captured 24 Sep 2026

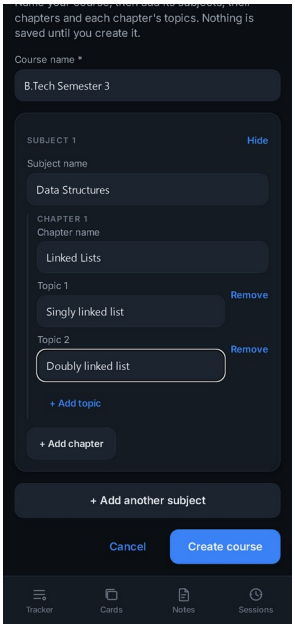


Figure 7.9: Build a course from scratch
captured 24 Sep 2026

7.3 Timers

Starting a timer from any row opens the timer sheet with Stopwatch, Pomodoro and Custom modes. Full screen turns the phone to landscape; a tap pauses or resumes, and while paused the student can pick one of six faces.

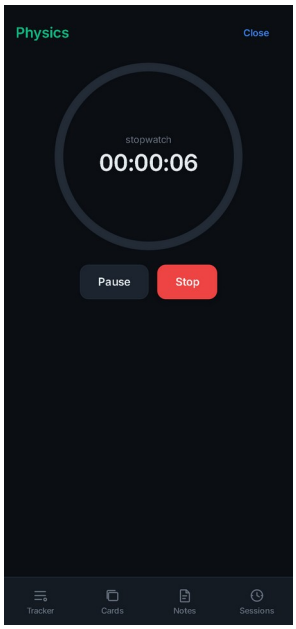


Figure 7.10: Timer sheet
captured 24 Sep 2026

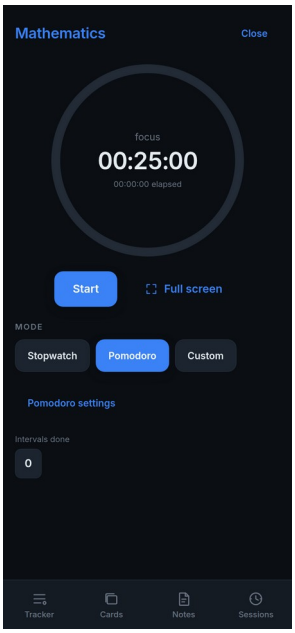


Figure 7.11: Pomodoro mode before starting
web demo, 6 Oct 2026

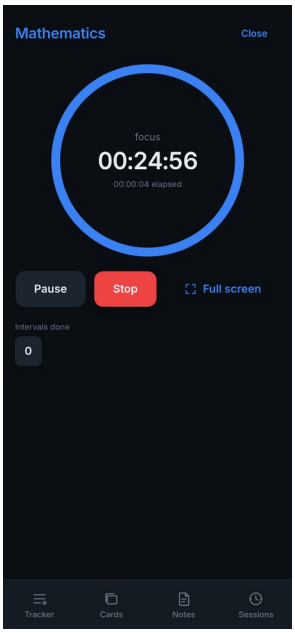


Figure 7.12: Pomodoro interval running
web demo, 6 Oct 2026

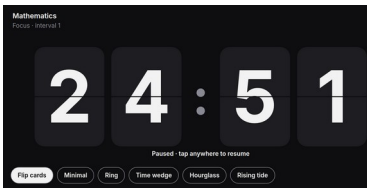


Figure 7.13: Full-screen timer, Flip cards face (paused)
web demo, 6 Oct 2026

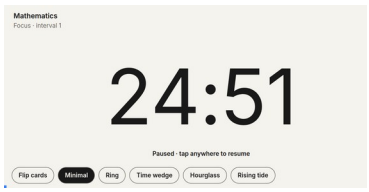


Figure 7.14: Minimal face
web demo, 6 Oct 2026



Figure 7.15: Ring face
web demo, 6 Oct 2026

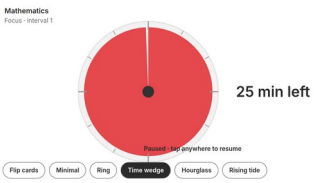


Figure 7.16: Time wedge face
web demo, 6 Oct 2026

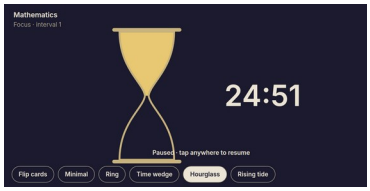


Figure 7.17: Hourglass face
web demo, 6 Oct 2026

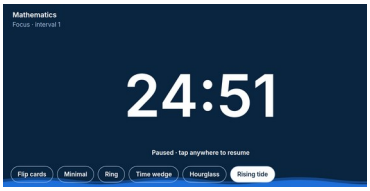


Figure 7.18: Rising tide face
web demo, 6 Oct 2026

7.4 Focus Mode

Focus Mode is a countdown of 1 to 240 minutes with presets, a choice of end alert and a keep-screen-on option. Its capability list says plainly what it cannot do on the current platform — the browser build marks screen-on and departure counting as app only, and app blocking as impossible.

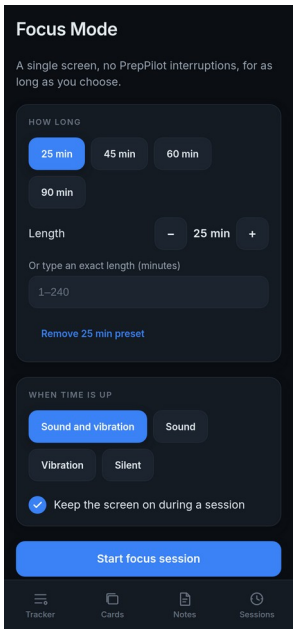


Figure 7.19: Focus Mode set-up
web demo, 6 Oct 2026

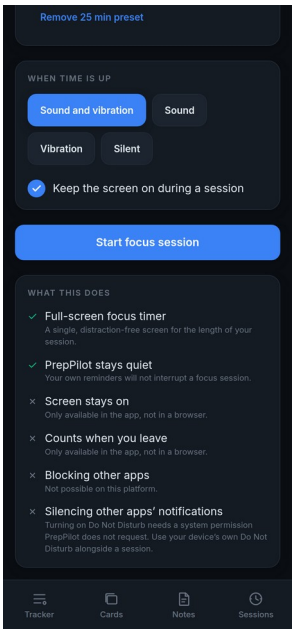


Figure 7.20: End alert and capability list
web demo, 6 Oct 2026

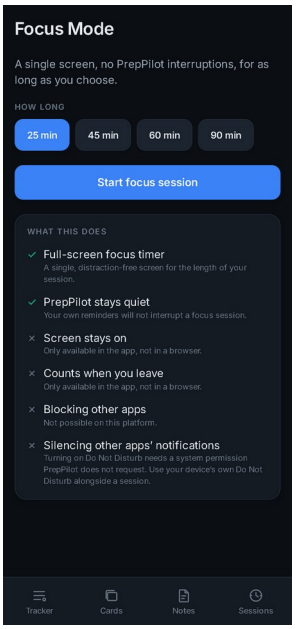


Figure 7.21: Focus Mode
captured 24 Sep 2026

7.5 Dashboard and sessions

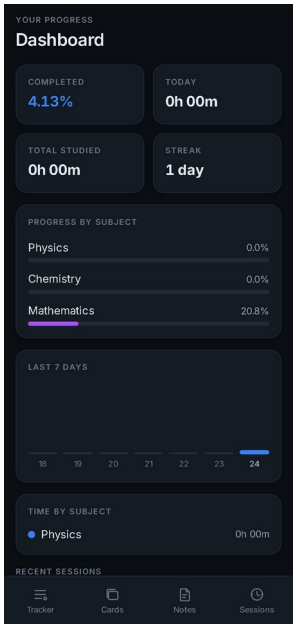


Figure 7.22: Dashboard
captured 24 Sep 2026

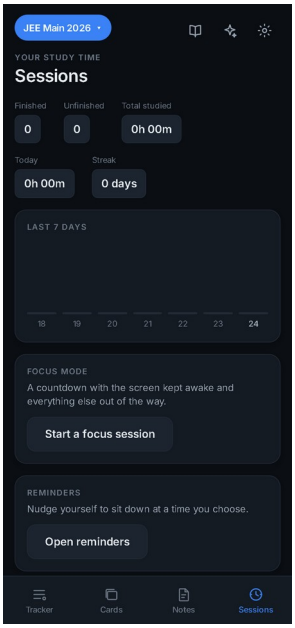


Figure 7.23: Sessions history
captured 24 Sep 2026

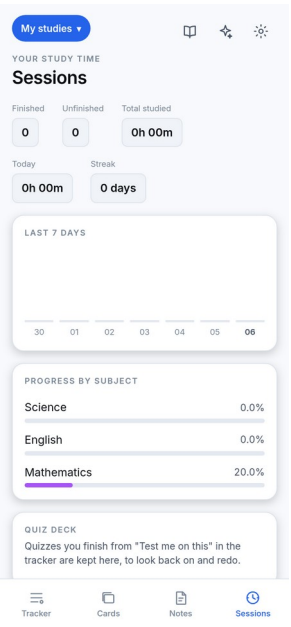


Figure 7.24: Sessions with Quiz deck, light mode
web demo, 6 Oct 2026

7.6 Revision, notes and reminders

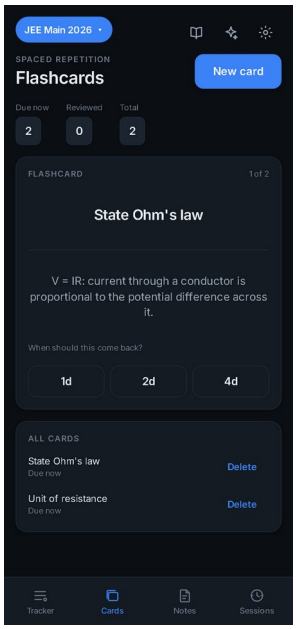


Figure 7.25: Flashcard review with 1d / 2d / 4d
captured 24 Sep 2026

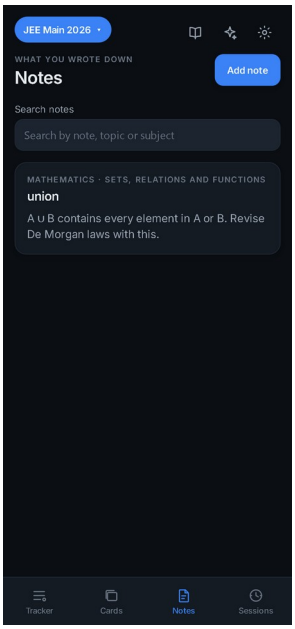


Figure 7.26: Notes by topic
captured 24 Sep 2026

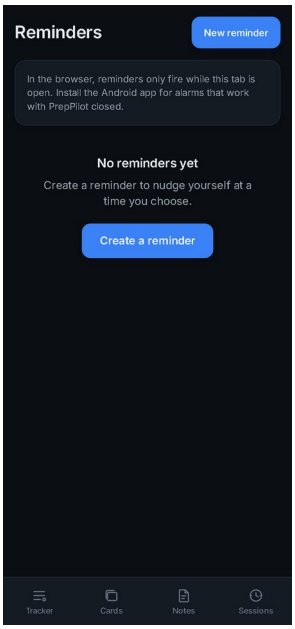


Figure 7.27: Reminders
captured 24 Sep 2026

7.7 AI assistant

The assistant answers questions, can read a photographed page, formats its replies, and turns a request for new material into a proposal the student must confirm.

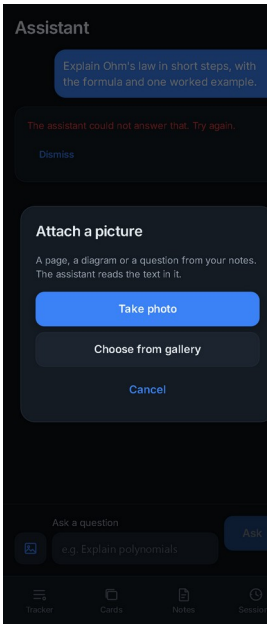


Figure 7.28: Attach a photo or screenshot
captured 24 Sep 2026

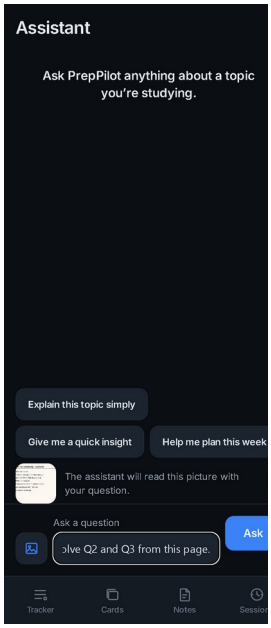


Figure 7.29: Picture attached to a question
captured 24 Sep 2026

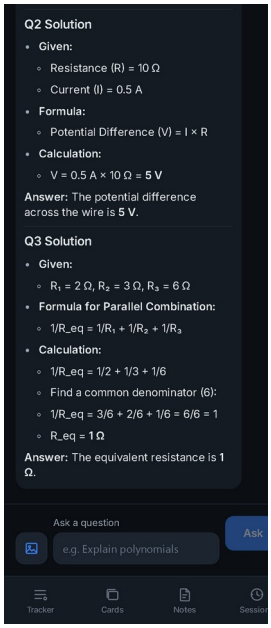


Figure 7.30: Answer about the picture
captured 24 Sep 2026

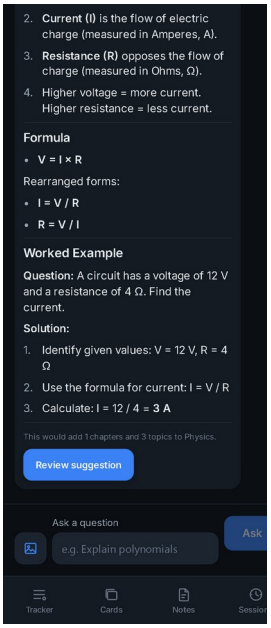


Figure 7.31: Formatted reply
captured 24 Sep 2026

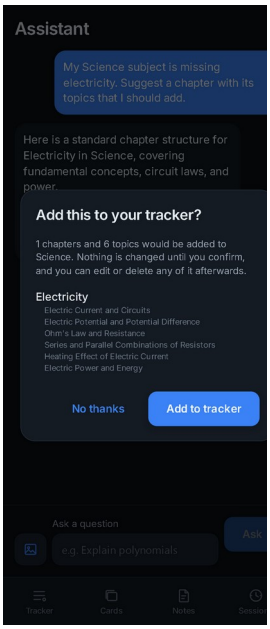


Figure 7.32: Proposal review before adding
captured 24 Sep 2026

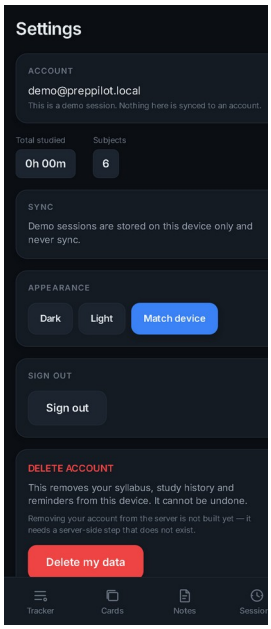


Figure 7.33: Settings
captured 24 Sep 2026

7.8 Downloadable themes

Settings → Themes lists Classic and three downloadable looks. Each downloads only when chosen (340–770 KB) and then works offline. For these captures the theme files were built from the repository and served locally.

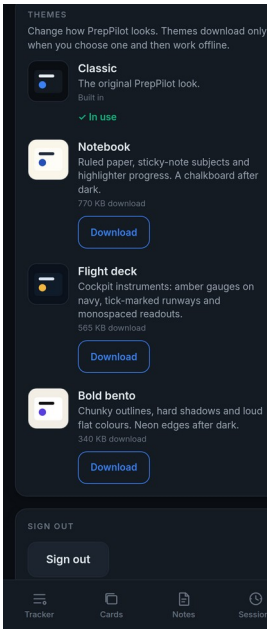


Figure 7.34: Theme list in Settings
web demo, 6 Oct 2026

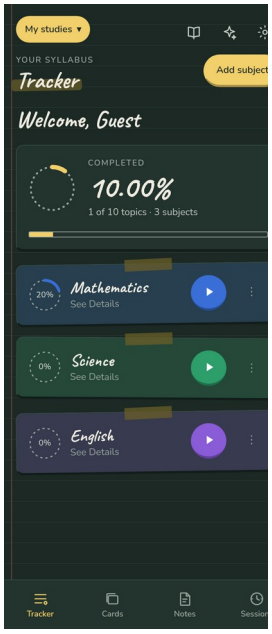


Figure 7.35: Notebook theme, dark
web demo, 6 Oct 2026

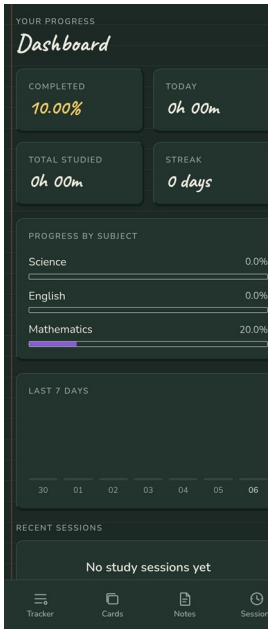


Figure 7.36: Notebook dashboard, dark
web demo, 6 Oct 2026

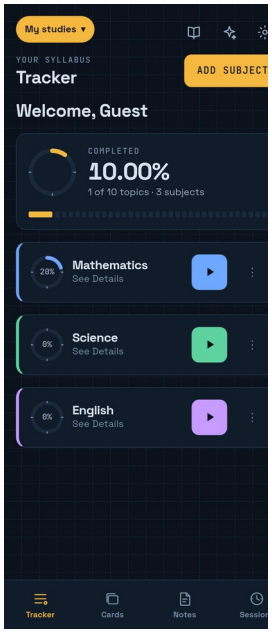


Figure 7.37: Flight deck theme, dark
web demo, 6 Oct 2026

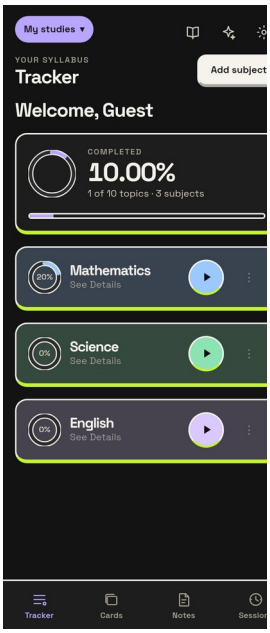


Figure 7.38: Bold bento theme, dark
web demo, 6 Oct 2026

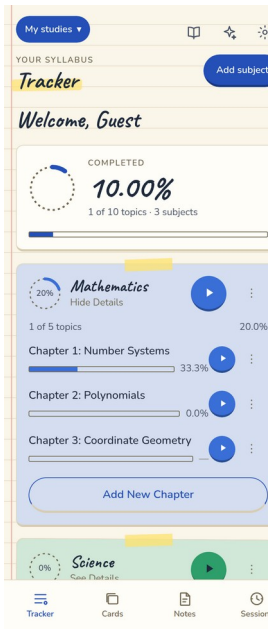


Figure 7.39: Notebook theme, light
web demo, 6 Oct 2026

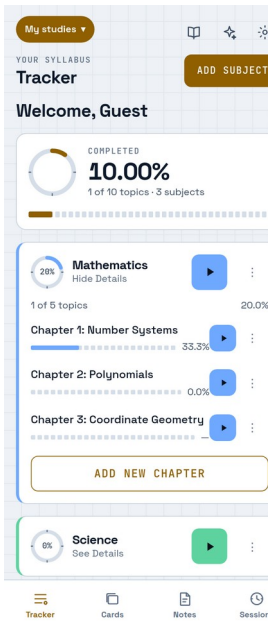


Figure 7.40: Flight deck theme, light
web demo, 6 Oct 2026

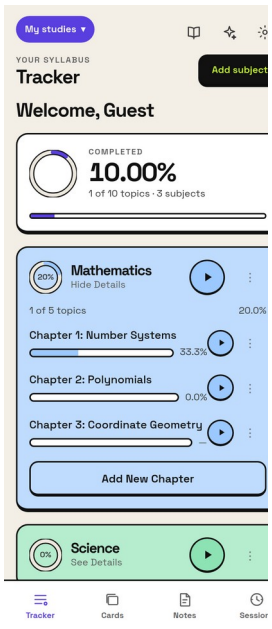


Figure 7.41: Bold bento theme, light
web demo, 6 Oct 2026

8. Testing and Validation

8.1 Strategy

Verification is automated. Pure logic in the shared package is tested with Vitest; the app's screens, stores and repositories with Jest and React Native Testing Library; and the Edge Functions with Deno's test runner, stubbing Gemini. Tests find controls by accessibility role and name rather than test IDs, so a control that a screen reader cannot reach fails the ordinary suite.

8.2 Test results

Table 8.1: Automated test results

Package	Runner	Suites	Tests	Result
apps/mobile	Jest	86	1,658	All passed (run 6 Oct 2026)
packages/shared	Vitest	17	573	All passed (run 6 Oct 2026)
supabase/functions	Deno test	2	52	Counted from source; run in CI, not re-run for this report
Total		105	2,283	

These figures are higher than the 2,073 tests recorded in the repository's README, which predates the themes and full-screen timer work.

8.3 What is tested against something real

- Repository tests run the **generated migrations** against in-memory SQLite (`better-sqlite3`), so they execute the SQL the app ships, including constraints and cascades.
- The sync engine runs the real local database against an in-memory server that behaves as PostgREST does, covering tombstones, rename-versus-tick, repeated passes and failed passes.
- The server schema was applied to a real PostgreSQL database and RLS isolation checked in both directions.
- Theme packs are checked against the validator, including WCAG contrast in both modes.

8.4 Continuous integration

Every push runs a GitHub Actions pipeline that checks formatting, type-checks every package, runs the tests with coverage thresholds (95% statements, 90% branches, 95% functions, 95% lines), runs the Edge Function tests, and verifies that both the Android and web bundles still build.

8.5 Device testing

The Android APK (about 41 MB, arm64) has been run on a physical phone against the live Supabase project: sign-in by email and by Google, restoring data after a reinstall, and the assistant including picture questions. This is recorded in the project's documentation as the developer's confirmation. Testing has not been systematic across Android versions, manufacturers' battery-optimisation policies or locales.

9. Results and Discussion

Table 9.1: Objectives against outcomes

Objective	Outcome
Accurate measurement	Achieved. Topic-weighted progress in one pure module, protected by a regression test; empty portions shown as —
Offline-first data	Achieved. Local SQLite is authoritative; automatic sync with deterministic merge; reinstall restore observed on the live project. A two-device conflict has been verified

	only by engine tests
Safe AI	Achieved. Key server-side only; proposals limited to additions and always reviewed; 200-topic cap
Integrated workflow	Achieved. Timers, flashcards, notes, quizzes and reminders all reference the syllabus

Beyond the original objectives, the delivered system adds photo questions to the assistant, saved conversations and follow-ups, a Quiz deck, a full-screen timer with six faces, a configurable Focus Mode and downloadable themes that do not enlarge the app download. Engineering decisions are recorded with their reasoning — entries D1–D110 in `DECISIONS.md` — so the system can be examined for why it was built as it was, not only for what it does.

10. Limitations

- **Flashcard scheduling is fixed-interval**, not SM-2, although SM-2 statistics are kept.
- **Focus Mode cannot block other apps or toggle Do Not Disturb** on Android; it says so in the app.
- **The local database is not encrypted.** The session token is in the keystore; the syllabus is not.
- **The AI rate limit is in memory** and resets when the function cold-starts.
- **The Gemini key is on the free tier**, shared by all users, and free-tier content may be used by Google; a long competitive quiz can take 30–60 seconds.
- **Only follow-up chats send earlier messages**; an ordinary chat sends each question alone. One picture per question; pictures do not sync.
- **Weekday reminders are re-armed by the app**, so they stop if the app is never opened.
- **Phone OTP** is implemented behind a flag but untested with a real SMS provider.
- **Sign-in is required once**: offline-first applies after the first successful sign-in.
- **A two-device conflict** has not been observed end to end on the hosted project.
- **No penetration test or security audit** has been carried out.
- **The APK is about 41 MB**; a Play Store app bundle would reduce the download.

11. Future Scope

- Switch flashcard review to adaptive SM-2 scheduling, using the statistics already kept.
- Move the AI rate limit into the database and log real token counts; move to Gemini's paid tier before real users arrive.
- Encrypt the local database.
- Systematic device testing across Android versions and manufacturers, and an observed two-device conflict test against the hosted project.
- Drag-to-reorder in the tracker (repositories already support reordering).
- Phone OTP with a real SMS provider; an iOS build; a Play Store release as an app bundle.
- More templates and community-contributed syllabi.

12. Conclusion

PrepPilot shows that a study tracker can be both offline-first and honest about progress. By making the device database authoritative, it works where students actually study; by computing progress topic-weighted and showing an empty portion as unmeasured, it gives students a figure they can plan with; and by keeping the AI key on the server and limiting the assistant to reviewed additions, it adds AI help without giving up control of the student's work. The system is implemented, runs on Android and in the browser, and is supported by more than two thousand automated tests and a documented record of its design decisions and limitations.

References

[1] H. Ebbinghaus, Memory: A Contribution to Experimental Psychology. New York: Teachers College, Columbia University, 1913 (orig. 1885).

[2] N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted and D. Rohrer, “Distributed practice in verbal recall tasks: A review and quantitative synthesis,” Psychological Bulletin, vol. 132, no. 3, pp. 354–380, 2006.

[3] P. A. Wozniak and E. J. Gorzelanczyk, “Optimization of repetition spacing in the practice of learning,” Acta Neurobiologiae Experimentalis, vol. 54, no. 1, pp. 59–62, 1994.

[4] M. Kleppmann, A. Wiggins, P. van Hardenberg and M. McGranaghan, “Local-first software: You own your data, in spite of the cloud,” in Proc. Onward! 2019, pp. 154–178.

[5] M. Shapiro, N. Preguiça, C. Baquero and M. Zawirski, “Conflict-free replicated data types,” in Proc. SSS 2011, pp. 386–400.

[6] F. Cirillo, The Pomodoro Technique. London: Virgin Books, 2018.

[7] Meta Platforms, “React Native documentation.” https://reactnative.dev/

[8] Expo, “Expo documentation.” https://docs.expo.dev/

[9] Supabase, “Supabase documentation.” https://supabase.com/docs

[10] Drizzle Team, “Drizzle ORM documentation.” https://orm.drizzle.team/

[11] Google, “Gemini API documentation.” https://ai.google.dev/docs

[12] K. Davis, B. Peabody and P. Leach, Universally Unique IDentifiers (UUIDs), IETF RFC 9562, 2024.

Appendix A: Glossary

Term	Meaning
Offline-first	The device database is the source of truth; the network is optional and sync happens later
Tombstone	A row marked deleted (deleted_at) instead of removed, so the deletion can be synchronised
UUIDv7	A time-ordered unique identifier generated on the device, so offline records have their final id at once
Row Level Security (RLS)	PostgreSQL rules attached to rows, so each user can read and write only their own data
Watermark	The time of the last successful sync; the next pull asks for rows changed since then
Edge Function	A small server-side function on Supabase, used here to hold the AI key and delete accounts
Proposal	An inert, parsed AI suggestion that can only add chapters or topics and needs confirmation
Demo mode	A browser build with a fixed local account and seeded data, no backend and no sync